

Garnet: A New Family of Hash Functions

Bin Zhang^{1,2,6,7}, Yingxin Li³, Zhenzhen Bao⁴, Wei Wang⁵, Rui Wang⁵, and Jianhui Zhang⁵

¹ Hefei National Laboratory, Hefei, China

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing, 100878, China,
`martin_zhangbin@hotmail.com`

³ East China Normal University, Shanghai, China
`yxli@sc.ecnu.edu.cn`

⁴ Tsinghua University, Beijing, China
`zzbao@mail.tsinghua.edu.cn`

⁵ University of Science and Technology of China, Hefei, China
`weiwang2025@ustc.edu.cn`, `{Kd220534,zhangjh3}@mail.ustc.edu.cn`

⁶ University of the Chinese Academy of Sciences, Beijing, 100049, China

⁷ Trusted Computing and Information Assurance Laboratory, SKLCS, Institute of Software, Chinese Academy of Sciences, Beijing, China

Abstract. In this paper, we present the Garnet family of hash functions, which could produce 512, 768 and 1024-bit digests according to the demand. The Garnet family of hash functions are designed to be efficient in both software and hardware. In the classical setting, the current security analysis shows that, for an n -bit digest, Garnet targets $n/2$ -bit collision resistance and n -bit preimage and second-preimage resistance. In the quantum query setting, our analysis has found no structural attack faster than the generic quantum attacks whose complexity is bounded by the digest length.

Keywords: Hash functions, Permutation-based, Garnet, Sponge, Quantum.

1 Background

The National Institute of Commercial Cryptography Standards (NICCS) of China has released a call for hash functions of large digest size [29] in March 2025, with the details available at <https://www.niccs.org.cn/>. It calls for hash functions that can generate 512/1024-bit digest, and optionally 768-bit digest respectively from the input message of maximal length of $2^{64} - 1$ bits. The instantiated hash functions should have the security level corresponding to their digest length against collision, preimage and second preimage attacks in both the classical and quantum models. This paper specifies three hash algorithms—Garnet-512, Garnet-768 and Garnet-1024a, respectively. The underlying permutations of these algorithms are designed to be efficient in both software and hardware with the parallel feature in mind and thus especially suitable for high-end environments. We adopt both the classical Sponge mode [2] and its variant the

Sponge-DM [33] which evaluates the permutation during the absorption phase in a Davies-Meyer mode, to construct the Garnet family of hash functions. We also generalize the original AES design methodology from bitwise to 128-bit words so that a large internal state can be handled in a parallel manner efficiently. As a result, the security of the Garnet hash functions with respect to differential cryptanalysis [4] can be estimated to have an appropriately high level under some reasonable assumptions.

The high degree of parallelism allows Garnet hash functions to run efficiently on the modern mainstream processors in software. With the new AES-NI instruction, the speed of Garnet (non-optimized) is about 8.87 cycles/byte on the Intel(R) Xeon(R) Gold 6242R CPU running 64-bit operating system with Microsoft Visual Studio C++ compiler. Some further optimized C/assembly codes of Garnet run at a faster speed whose throughput is well above 1000.00 Mbps under the benchmark Ubuntu 24.04 operating system [29] using GCC compiler. Besides, we can expect some hash speed of Garnet below 4.00 cycles/byte by the Intel C++ compiler 2025. The memory required for the hardware implementation of Garnet hash functions is 2048 bits with the round constants accessed on the fly. Garnet-512, Garnet-768 and Garnet-1024a share the same internal state structure and size, so it is very efficient to implement these hash algorithms together in hardware. Based on the ideal model assumption in Section 4, the current cryptanalysis of Garnet shows that it could provide the security that is consistent with the theoretical bounds according to the digest length. For an input message block, it will pass through at least 16-round permutation applications that involves 4096 8-bit AES Sboxes. We found that a differential characteristic trail in such a 16-round compression function involves more than 400 active Sboxes under some reasonable assumptions. In fact, the differential behaviour of Garnet will be more complicated than the considered case and it is expected that there are even more number of differentially active Sboxes than the lower bounds obtained so far due to the complex interactions introduced by the designed diffusion transforms in different directions of the internal state. If we model the precise bitwise behaviour of differential propagation by taking into account the comprehensive transforms, we will have a Mixed Integer Linear Programming (MILP) or Boolean Satisfiability Problem (SAT) model that is too complex to be solved in a reasonable time by the state-of-the-art solvers. Thus, it seems safe to conclude that the large number of differentially active Sboxes of Garnet ensures enough resistance against differential-based attacks. As a result of this finding, we have made a comprehensive security analysis of Garnet which covers all the known cryptanalytical techniques against hash functions and the cryptanalysis results seem to indicate that the current design reaches the theoretical bounds determined by the digest length in both the classical and quantum settings.

This paper is organized as follows. The specifications of Garnet are given in Section 2 with the different algorithmic instances. Section 3 describes the design rationale behind Garnet from the algorithmic structure to the concrete components with the specified parameter choices arguments. The security statements

of Garnet and the companion security analysis of Garnet is presented in Section 4. The performance evaluation of Garnet is provided in Section 5, while the features statements are pointed out in Section 6. Finally, Section 7 concludes this paper.

2 Specifications of Garnet and its Variants

In this section, we present the detailed description of Garnet hash functions. From some pre-defined 128-bit constants and algorithmic parameters, Garnet hash functions can generate the 512-bit, 768-bit and 1024-bit digest for the input message of maximal length of $2^{64} - 1$ bits.

2.1 Preliminaries

We adopt the following notations throughout this paper.

- $\oplus$: bitwise exclusive or, which is usually executed at the 128-bit word level
- $+$: addition over the finite field $\text{GF}(q)$, which is identical with the $\oplus$ operation if $q = 2^v$ for some integer v
- S : the state of Garnet, which consists of 16 128-bit words $S = (S_0, S_1, \dots, S_{15})$ for Garnet-512, Garnet-768 and Garnet-1024a, where S_i ($0 \leq i \leq 15$) is the i -th 128-bit word and S_0 is the first word
- The i -th word at round r is defined as S_i^r and the j -th bit of a word S_i is denoted by $S_{i,j}$, thus $S_{i,j}^r$ is the j -th bit of the i -th word at round r
- M_i is a 128-bit message word after padding
- H_i is a 128-bit hash value block
- $mlength$: the bit-length of the input message M before padding with $|M| = mlength$
- $x \mid y$: the integer y is a multiple of the integer x
- $const_0$: a 128-bit constant, which is 0xc90fdaa22168c234c4c6628b80dc1cd1
- $const_1$: a 128-bit constant, which is 0x29024e088a67cc74020bbea63b139b22
- $const_2$: a 128-bit constant, which is 0x514a08798e3404ddef9519b3cd3a431b
- $const_3$: a 128-bit constant, which is 0x302b0a6df25f14374fe1356d6d51c245
- $counter_0$: a 64-bit constant, which is 0x000002000e0a0e0a
- $counter_1$: a 64-bit constant, which is 0x000002800e0a0e0a
- $counter_2$: a 64-bit constant, which is 0x000003000e0a0e0a
- $counter_3$: a 64-bit constant, which is 0x000003800e0a0e0a
- $counter_4$: a 64-bit constant, which is 0x000004000e0a0e0a
- $counter_5$: a 64-bit constant, which is 0x000002000f0b0f0b

- $counter_6$: a 64-bit constant, which is 0x00008380100c100c

The four 128-bit constants $const_i$ ($0 \leq i \leq 3$) are derived from the first 512 bits of the binary representation of π and the counter values $counter_i$ ($0 \leq i \leq 6$) are used in the initial state of Garnet to specify the algorithmic choices including the iteration mode, the rate r , the round numbers in the initialization phase, the absorbing phase, middle processing phase and squeeze phase, respectively. The 47th most significant bit of a 64-bit counter is used to indicate the iteration mode, i.e., for Sponge mode [2] the msb is 0; while for Sponge-DM mode [33], it is 1. For example, $counter_0 = 0x000002000e0a0e0a$ means that the rate r in Sponge mode is $0x0200 = 512$, the initialization phase has $0x0e = 14$ rounds, the absorbing phase has $0x0a = 10$ rounds, the middle processing phase has $0x0e = 14$ rounds and the squeeze phase has $0x0a = 10$ rounds. Similarly, $counter_6 = 0x00008380100c100c$ means the rate r in Sponge-DM mode is $0x0380 = 896$, the initialization phase has $0x10 = 16$ rounds, the absorbing phase has $0x0c = 12$ rounds, the middle processing phase has $0x10 = 16$ rounds and the squeeze phase has $0x0c = 12$ rounds.

2.2 The Message Padding

For the input message M that is to be hashed by Garnet, we need to first pad the message so that its length reaches a multiple of 128 bits. It is then grouped into w -bit blocks with $128 \mid w$ to be processed iteratively by Garnet.

Unless explicitly stated otherwise, Garnet adopts the following canonical bit/word serialization convention. The input bits are consumed in a most-significant-bit first manner inside each input byte. Each 128-bit word is represented as a little-endian 16-byte integer, i.e., the 0th byte is the least significant byte of the 128-bit word. Within a rate block, the first serialized 128-bit message word is absorbed into the lowest-numbered selected state word, and the subsequent serialized message words are absorbed into the selected state words in an increasing order according to the state-word index. The digest extraction emits the selected state words in an increasing order according to the selected state-word.

The concrete message padding rule works as follows. It appends a single 1 and the smallest number of 0s to the last block of the message M such that the length of the padded message is a multiple of 128 bits. The resulting padded message is split into m blocks of 128 bits, i.e.,

$$\begin{aligned} M &= M_0 || M_1 || \dots || M_{m-1}, M_{m-1} = * * \dots *, * || 1 || 0^{127 - (|M| \bmod 128)} \\ &= * * \dots *, * || 1 || 0^{127 - (m \cdot length \bmod 128)}, \end{aligned} \quad (1)$$

where we denote the bit-length of the original un-padded message M by $m \cdot length$ as defined in section 2.1. For simplicity and without confusion, we also denote the message after padding by M . The message block of w -bit processed in the absorbing phase is xored to the rate r part of the internal state S , followed by an application of the b -round permutation P^b to the whole state S . For different variants of the Garnet algorithm, the corresponding choices of w , r and b are different, which are specified later.

2.3 The State and the Initialization

The Garnet family of hash functions has an unkeyed permutation working on a $128 \cdot \gamma$ bits internal state S for $\gamma = 16$, which are arranged as $d \times d = 4 \times 4$ matrices shown below

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

where S_i is the 128-bit word for $0 \leq i \leq 15$, as defined in Section 2.1. The permutation P is applied on the internal state S of d^2 elements of 128-bit each, which are represented as a $d \times d$ matrix for $d = 4$ as depicted above. For $d = 4$, the layout of the initial state S^0 in the initialization phase is

c_0	0	0	0
0	c_1	0	0
0	0	c_2	t_0
t_1	0	0	c_3

where $c_i = \text{const}_i$ ($i = 0, 1, 2, 3$) are the four 128-bit constants specified in Section 2.1, which are the first 512 bits in the binary representation of π , t_0 is the bit-length of the generated digest and t_1 is a counter whose value follows the pattern defined in Section 2.1 and chosen according to the concrete algorithmic variant of Garnet.

After the loading of the initial state S^0 , the a -round permutation P^a is applied to get the starting state of the absorbing phase as $S^a \leftarrow P^a(S^0)$, which can be pre-computed for each Garnet algorithm instance. We get, for instance, the starting state of the absorbing phase for Garnet-512 with *counter*₀, as shown below.

$$\begin{aligned}
S_0^a &= \text{0xde30d1b89ddaa011782b1463e5b448c2} \\
S_1^a &= \text{0x634ee0b3bf88f484d92183bde01eaf39} \\
S_2^a &= \text{0x149d3ee603859d70f8fb07c0f0d96f62} \\
S_3^a &= \text{0xf48dce216aa60cac4c92e211237707ef} \\
S_4^a &= \text{0x7396d12e4c6dbbb7d2543469c3db12b7} \\
S_5^a &= \text{0x8a6a75f1e6bbc9596d127c0b76361e62}
\end{aligned}$$

$$\begin{aligned}
S_6^a &= 0x530cf1efa5661a70833bfb64f3bce994 \\
S \leftarrow S_7^a &= 0xb0389a5d5e9c99700e267b18c6f6db4c \\
S_8^a &= 0x67117fbd723084d205239d1166b906cb \\
S_9^a &= 0xcbbac776493d5c27e907c5dd6ca25474 \\
S_{10}^a &= 0xaff5e27600e5bc3f6d0064b8f5707068 \\
S_{11}^a &= 0x9b1ac6e440a1ef11c58584622d4b401e \\
S_{12}^a &= 0x92918651c356f4b842e77c2f23ac28fb \\
S_{13}^a &= 0x7e5c987ec397303609e68e582255f277 \\
S_{14}^a &= 0x8b07f08ed4e24d6c87947a0718e95f65 \\
S_{15}^a &= 0xfa7f68f3b89fc0c7f742ee05141a3f41 .
\end{aligned}$$

For other pre-computed initial states, we list them in Appendix A.

2.4 The Permutation

Once the state S is initialized, we can apply the permutation P iteratively to S in each phase of the Garnet algorithm, given the padded input message M . One round of the internal permutation P consists of four transforms, i.e.,

$$P = \zeta \circ \theta \circ \rho \circ \sigma,$$

which are defined as follows.

- σ (**SubWord**): $\text{AESL}(S_i, B)$ is applied in parallel to each S_i ($0 \leq i \leq 15$) of the full state S , where $\text{AESL}(A, B) = \text{MixColumns} \circ \text{SubBytes} \circ \text{ShiftRows}(A, B)$ is the AES-NI instruction with A, B being the two 16-byte input words corresponding to the 16-byte state and 16-byte round key, respectively. This instruction fulfills one round of AES encryption except the last round [13]. It is worth noting that the (vectorial) AES-NI instructions are available on all the mainstream CPU microprocessors including x86 and ARM processors. For example, on x86 processors, this function has been efficiently implemented and can be called via `_mm_aesenc_si128(A, B)`, where A and B are two 128-bit integers `_m128i`. σ is the only non-linear function in the permutation P . It is interesting to note that any permutation over $\text{GF}(2)^{128}$ with some close differential and linear cryptographic properties to $\text{AESL}(S_i, B)$ can take the role of σ in the construction.
- ρ (**MixColumn**): The **MixColumn** step is a linear transform operating on the state S column by column over $\text{GF}(2^{128})$ in parallel. Instead of operating on 4-byte columns like in AES [13], the ρ function works on 4 128-bit words in the same column of the underlying state matrix of size 4×4 . The **MixColumn** function provides the global diffusion in the permutation P in the column direction, which can be implemented as four matrix multiplications over $\text{GF}(2^{128})$. For $d = 4$, for each column λ_i ($i = 0, 1, 2, 3$) of the state matrix

S , we apply the following MDS matrix T_1 to each column of S . Concretely, $\lambda_i \leftarrow T_1 \cdot \lambda_i$ with

$$T_1 = \begin{pmatrix} \alpha & \alpha+1 & 1 & 1 \\ 1 & \alpha & \alpha+1 & 1 \\ 1 & 1 & \alpha & \alpha+1 \\ \alpha+1 & 1 & 1 & \alpha \end{pmatrix},$$

where α is the primitive element of the finite field $\text{GF}(2^{128})$, which is defined as $\text{GF}(2)[x]/(x^{128} + x^7 + x^2 + x + 1)$ and the polynomial $x^{128} + x^7 + x^2 + x + 1$ is a primitive polynomial over $\text{GF}(2)$. For $d = 4$, the four columns of the state matrix are

$$\begin{aligned} \lambda_0 &= (S_0, S_4, S_8, S_{12}) & \lambda_1 &= (S_1, S_5, S_9, S_{13}) \\ \lambda_2 &= (S_2, S_6, S_{10}, S_{14}) & \lambda_3 &= (S_3, S_7, S_{11}, S_{15}). \end{aligned}$$

Table 1. The round constants for the $d = 4$ case

Round N_r	$d = 4$ ($c_0^{(r)}, c_1^{(r)}, c_2^{(r)}$)
1	1, 1, 2
2	3, 5, 8
3	13, 21, 34
4	55, 89, 144
5	233, 377, 610
6	987, 1597, 2584
7	4181, 6765, 10946
8	17711, 28657, 46368
9	75025, 121393, 196418
10	317811, 514229, 832040
11	1346269, 2178309, 3524578
12	5702887, 9227465, 14930352
13	24157817, 39088169, 63245986
14	102334155, 165580141, 267914296
15	433494437, 701408733, 1134903170
16	1836311903, 2971215073, 4807526976

- θ (**AddConstant**): The constant addition function θ modifies the state S with the bitwise xor operations executed at some special positions of S . The constants have been chosen such that each of the N_r round computations are different, and such that the classical symmetry between the columns of the state matrix will be destroyed. Currently, the round constants can be generated by the classical Fibonacci sequence, as shown in the Tables 1 for the $d = 4$ case. For performance reasons, only the last column (the rightmost column) of the internal state S is involved in the θ function. Precisely, for $d =$

- 4, the state S is modified as $(S_7^r, S_{11}^r, S_{15}^r) \leftarrow (S_7^r \oplus c_0^{(r)}, S_{11}^r \oplus c_1^{(r)}, S_{15}^r \oplus c_2^{(r)})$ for round r , where $c_i^{(r)}$ ($1 \leq r \leq 16$) ($0 \leq i \leq 2$) is the round-dependent constants generated by the Fibonacci sequence, as shown in the Table 1.
- ζ (**MixDiagonal**): The **MixDiagonal** function ζ is another linear transform operating on the state S diagonal by diagonal over $\text{GF}(2^{128})$ in parallel. This function provides the global diffusion in a different direction from the **MixColumn** function ρ , thus further mix the words in the state S . For $d = 4$, for each diagonal η_i ($i = 0, 1, 2, 3$) of the state matrix S , we apply the following MDS matrix T_2 to each diagonal of the internal state S . Concretely, $\eta_i \leftarrow T_2 \cdot \eta_i$ with

$$T_2 = \begin{pmatrix} \alpha & \alpha + 1 & 1 & 1 \\ 1 & \alpha & \alpha + 1 & 1 \\ 1 & 1 & \alpha & \alpha + 1 \\ \alpha + 1 & 1 & 1 & \alpha \end{pmatrix},$$

where α is the primitive element of the finite field $\text{GF}(2^{128})$ defined as $\text{GF}(2)[x]/(x^{128} + x^{29} + x^{15} + x^2 + 1)$ and the polynomial $x^{128} + x^{29} + x^{15} + x^2 + 1$ is another primitive polynomial over $\text{GF}(2)$. For $d = 4$, the four diagonals of the state matrix S are

$$\begin{aligned} \eta_0 &= (S_0, S_5, S_{10}, S_{15}) & \eta_1 &= (S_1, S_6, S_{11}, S_{12}) \\ \eta_2 &= (S_2, S_7, S_8, S_{13}) & \eta_3 &= (S_3, S_4, S_9, S_{14}). \end{aligned}$$

ζ can also be implemented as d matrix multiplications over $\text{GF}(2^{128})$ defined by the corresponding primitive polynomial over $\text{GF}(2)$. It is worth noting that the different definitions of the finite field $\text{GF}(2^{128})$ will allow several different ways of opening the byte-wise boundary in the hash computation process.

2.5 Hash Construction Mode

Once the input message M is padded following the rules in Section 2.2, Garnet will process the message M block by block according to some iteration mode, where the permutation P is applied to the internal state S to compute the hash value. Let the w -bit blocks processed in Garnet be

$$M = M_0 || M_1 || \cdots || M_{m-1} = M'_0 || M'_1 || \cdots || M'_{m'-1}, \quad (2)$$

where M'_j ($0 \leq j \leq m' - 1$) being the w -bit block that concatenates $\frac{w}{128}$ 128-bit blocks and $M'_{m'-1}$ is the last w -bit block consisting of the last padded 128-bit block M_{m-1} with the subsequent bits in the same w -bit block being 0s. We suggest to use two iteration modes in Garnet. One is the classical Sponge construction, as shown in Fig.1. The other is the recently proposed Sponge-DM mode in [33], as shown in Fig.2. It is worth noting that the Sponge mode in Garnet is a little bit different from the original Sponge mode in [2], where there is a P^a operation in the middle processing phase to process $mlength$, the

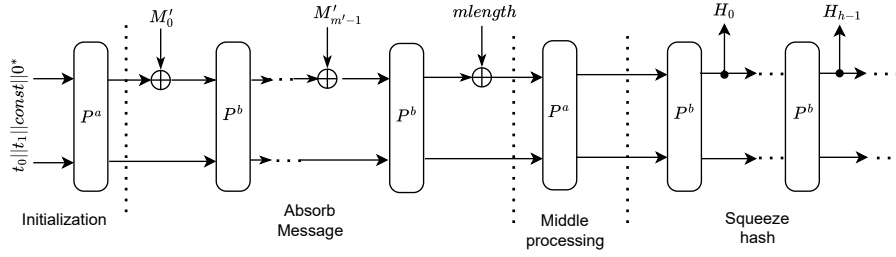


Fig. 1. The Sponge Mode in Garnet

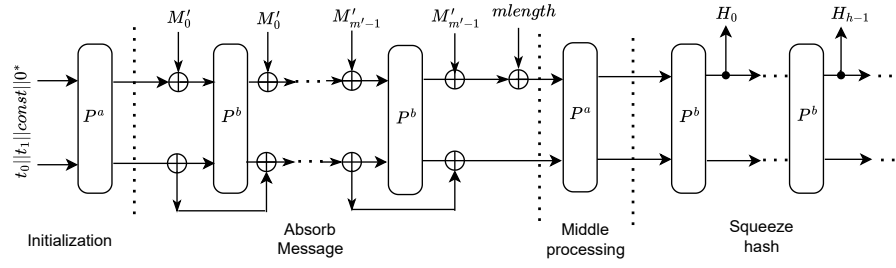


Fig. 2. The Sponge-DM Mode in Garnet

bit-length of the original un-padded message and a P^b operation next to P^a before the first extracting of the hash values in Fig.1. The same difference exists between the Sponge-DM mode in Garnet and the original Sponge-DM mode in [33], as depicted in Fig.2.

For implementation conformance, the two iteration modes are defined as follows. Let $I_r(X)$ denote the b -bit state value obtained by placing the r -bit rate block X into the selected rate cells and filling all remaining cells by zero. In the Sponge mode, the absorbing update is

$$S^{j+1} = P^b(S^j \oplus I_r(M'_j)), \quad 0 \leq j < m'.$$

After all message blocks have been absorbed, $mlength$ is injected into the rate part in the same serialized convention, followed by the middle processing and

Table 2. The security bounds of Sponge and Sponge-DM modes with digest size n , input rate r and output rate r'

Mode	Security			
	Indiff.	Collision	Preimage	2nd preimage
Sponge	$\frac{c}{2}$	$\min\{\frac{n}{2}, \frac{c}{2}\}$	$\min\{n, \max\{n - r', \frac{c}{2}\}\}$	$\min\{n, \frac{c}{2}\}$
Sponge-DM	$\frac{c}{2}$	$\min\{\frac{n}{2}, \frac{c}{2}\}$	n	$\min\{n, c - \log_2(\alpha)\}$

the pre-squeezing permutation. In Sponge-DM mode, the absorbing update is the full-state Davies-Meyer update

$$X_j = S^j \oplus I_r(M'_j), \quad S^{j+1} = P^b(X_j) \oplus X_j, \quad 0 \leq j < m'.$$

The feed-forward is applied to the complete internal state after every absorbed block, including the final padded block.

We have the following two Propositions on the provable security bounds of the two iteration modes in Garnet with the corresponding provable security bounds of the original Sponge and Sponge-DM modes shown in Table 2. In Table 2, all parameters are expressed in bits and the constants and logarithmic factors in b or n are ignored. The bounds assume that $c' \geq c/2 + \log_2(r)$, and α denotes the maximum message length in blocks. The output size n is assumed to satisfy $n \leq b$, ensuring the existence of a preimage with high probability.

Proposition 1. *The Sponge mode in Garnet inherits the same provable security bounds as the original Sponge mode.*

Proof (sketch). The indistinguishability proof of [2] models the underlying primitive as an ideal (uniformly random) permutation that the adversary may query in both the forward and inverse directions, and the resulting bounds are insensitive to the concrete number of rounds, since each phase is merely a fixed number of applications of that ideal permutation. The Sponge mode in Garnet preserves the structural ingredients on which the original theoretical proof in [2] relies: a proper injective padding, a fixed capacity that is never directly addressed by the message, and a fixed-length output projection. The additional P^a (length-injection) and P^b (pre-squeezing) operations are themselves further applications of the ideal permutation and therefore do not weaken the bounds. We emphasise that this argument is a reduction to the *ideal-permutation* model; the different round-key values used for domain separation across phases are modelled as a labelled family of independent ideal permutations. Establishing that the concrete 10–16-round Garnet permutation behaves as such an ideal permutation is the object of the dedicated cryptanalysis in Section 4 and forms an assumption of the current theorem. $\square$

Similarly, we have the following Proposition on the Sponge-DM mode in Garnet.

Proposition 2. *The Sponge-DM mode in Garnet inherits the same provable security bounds as the original Sponge-DM mode.*

Proof (sketch). The Sponge-DM proof in [33] additionally requires the full-state Davies-Meyer feed-forward as specified in Fig.2, namely $S_{j+1} = P^b(X_j) \oplus X_j$ for every absorbed block and for the complete state value X_j . Under the ideal-permutation model and the stated restriction on c' , this is exactly the structural condition used in the Sponge-DM bound of Table 2. The final length injection, domain-separated middle processing, and final output projection do not expose a chaining value that can be continued by an adversary without knowing the hidden capacity state. As in the Sponge mode in Garnet in Proposition 1, this

is an ideal model statement, based on the assumption that the concrete permutations exploited in the hash construction model is random. $\square$

Now we specify the concrete number of rounds for each operation phase in Figs.1 and 2 respectively, which is depicted in Table 3 with P^i meaning i rounds of application of the permutation P to the internal state S in the corresponding Garnet variant. It is worth noting that Garnet-1024a works on the 4×4 internal state S of 2048 bits with the Sponge-DM mode in Fig.2. Besides, to be compatible with both Tables 3 and 2, the maximal rate r of the Sponge-DM mode is $r_{\max} = 960$ bits, given the maximal message length of $2^{64} - 1$ bits.

Table 3. The concrete parameters for the Sponge and Sponge-DM modes. The rate r (in bits) and the capacity $c = b - r$ (with state size $b = 2048$ bits) listed here are the *normative* values of each instance; Garnet-512 also admit the optional rate variants encoded by the other *counter_i* (see Appendix B), whereas Garnet-768 uses the single rate $r = 512$.

Algorithm	Initialization	Absorb message	Middle processing	Squeeze hash
Garnet-512	P^{14}	P^{10}	P^{14}	P^{10}
Garnet-768	P^{15}	P^{11}	P^{15}	P^{11}
Garnet-1024a	P^{16}	P^{12}	P^{16}	P^{12}

Further, in order to make the domain separation among the different phases in the two iteration modes, different round-keys for the non-linear σ transform are adopted. For example, in Garnet-512, we use $B = 0$ in the $\text{AESL}(S_i, B)$ instruction when absorbing the message blocks, $B = 1$ in the middle processing phase and $B = 2$ for Squeezing hash values. Table 4 depicts the configuration for each variant algorithm in the σ transform when making the domain separation in Garnet.

Table 4. The domain separation in σ for the Sponge and Sponge-DM Modes

Algorithm	Initialization	Absorb message	Middle processing	Squeeze hash
Garnet-512	$B = 0$	$B = 0$	$B = 1$	$B = 2$
Garnet-768	$B = 0$	$B = 0$	$B = 3$	$B = 4$
Garnet-1024a	$B = 0$	$B = 0$	$B = 7$	$B = 8$

2.6 Absorbing and Squeezing

From the iteration modes in Figs.1 and 2, there will be some message absorbing operation at the beginning of each P^b application. Since there is some constraint

conditions imposed by the provable security bounds in Table 2, we specify the concrete positions where the w -bit message block is absorbed in the internal state S for each Garnet variant algorithm below, where the grey cells indicate the absorbing word positions given the corresponding rate of the Garnet instance.

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-512/ $w=r=512$ bit

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-512/ $w=r=1024$ bit

The absorbing word positions of two Garnet-512 instances with $r = 512$ and $r = 1024$ bits are illustrated above. For the other possible values of the rate r in Garnet-512 and Garnet-1024a, we include them in Appendix B.

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-768/ $w=r=512$ bit

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-1024a/ $w=r=896$ bit

The absorbing patterns of Garnet-768 and Garnet-1024a are depicted above. Precisely, the message absorbing pattern for each Garnet variant algorithm is as follows. For any selected grey cells (S_i, S_j) satisfying $i < j$ in the w -bit message block, let S_i stay more closely to the least significant side. For example, in Garnet-512 with $w = r = 512$ bits, the w -bit message block $M_i = M_{i,3} || M_{i,2} || M_{i,1} || M_{i,0} = S_{15} || S_{10} || S_5 || S_0$, where each $M_{i,j}$ ($0 \leq j \leq 3$) is of 128 bits after padding and $M_{i,3}$ is the most significant word.

When Squeezing the hash values in the Sponge and Sponge-DM modes in Figs.1 and 2 in Garnet, we have different extracting patterns for each variant algorithm in the Garnet family as shown below, where the squeezing pattern of Garnet-512 cover all the input rate cases.

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-512

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-768

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-1024a

The final digest is generated according to the above extracting pattern for each algorithm and for any selected grey cells (S_i, S_j) satisfying $i < j$, let S_i stay more closely to the least significant side. Precisely, we have

Garnet-512 The digest H is given as $H = S_{14}||S_9||S_7||S_0$, where S_0 is the least significant 128 bits and S_{14} is the most significant 128 bits.

Garnet-768 The digest H is given as $H = S_{15}||S_{13}||S_9||S_6||S_3||S_0$, where S_0 is the least significant 128 bits and S_{15} is the most significant 128 bits.

Garnet-1024a The digest H is given as $H = S_{14}||S_{12}||S_{11}||S_9||S_7||S_5||S_2||S_0$, where S_0 is the least significant 128 bits and S_{14} is the most significant 128 bits.

In summary, the Garnet family of hash functions consists of five steps: padding the input message M (Section 2.2), grouping the padded message into w -bit message blocks (Section 2.5), loading the pre-computed starting state according to the concrete Garnet instance (Section 2.6), computing the final hash value according to the iteration mode (Section 2.5), and generating the final digest following the squeezing pattern (Section 2.6).

3 Design Rationale

In this section we present the design principles behind Garnet. The general principle we follow in the design is security, efficiency and simplicity. We explain what we mean by it and how it is achieved from the designers' viewpoint.

Garnet is designed to achieve the following goals:

1. Simple
2. Secure

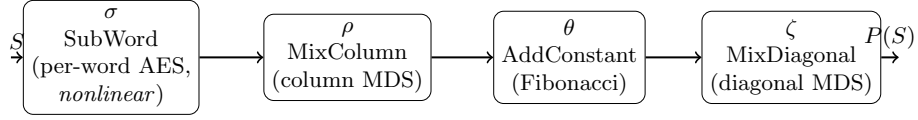


Fig. 3. One round of the Garnet permutation $P = \zeta \circ \theta \circ \rho \circ \sigma$. Only σ is non-linear; ρ and ζ are two MDS layers diffusing in the column and diagonal directions, respectively, and together they achieve full diffusion of the $d \times d$ state within a single round.

3. Efficient in both software and hardware
4. Scalability

The overall structure of the Garnet family of hash functions is very simple and the corresponding cryptanalysis is also easy to fulfill based on some reasonable assumptions due to the simplicity of the state structure. Due to the simple yet fast diffusion, the cryptanalysis based on the known techniques will not work for a sufficient large number of rounds. This clearly contributes to the cryptographic strength of the whole design. The permutation $P = \zeta \circ \theta \circ \rho \circ \sigma$ is constructed by a combination of local non-linear transform σ and two global transforms ρ and ζ working over two different extension fields $\text{GF}(2^{128})$. Thus, Garnet can achieve full 128-bit word diffusion within one round because of the parallel application of ρ in the column direction and the parallel application of ζ in the diagonal direction after ρ . Garnet is very efficient in both software and hardware due to the simple design strategy and the parallel nature, as can be see later in Section 5. Finally, The internal state of Garnet is large enough to hold some future additional input tweaks into the initial state S^0 , thus providing the flexible scalability in the design.

3.1 The Generalized AES Structure

To meet the requirement of 512/768/1024 bits security, we need a large internal state S to include both the rate r and the capacity c that should be consistent with the provable security bounds in Table 2. This motivates us to select the matrix structure of $d \times d$ 128-bit words depicted in Section 2.3, which will provide enough space for the 512/768/1024 bits security levels when embedded into the Sponge and the Sponge-DM modes. One round of the resulting permutation is depicted in Fig. 3. As depicted in Fig.3, Garnet has the properties that the full 128-bit word level diffusion will be achieved within one round of the permutation P for the 4×4 internal state S .

3.2 Round number

The round number shown in Table 3 is chosen according to the differentially active S-boxes analysis in Section 4.1. It is easy to see that Garnet adopts some conservative choice on the round number to maintain a comfortable security margin against future cryptanalysis. For different Garnet instances, we adopt

some round number that will assure the increasing security level by taking into account the security analysis results in Section 4.

The whole design of Garnet obtains a good tradeoff between security, efficiency and versatility by adopting of parallelism and the concrete choices of the various operations in the permutation P , including the corresponding number of rounds in each phase in the Sponge and Sponge-DM modes.

3.3 The Diffusion Components

We adopt two different MDS matrices of size 4×4 over $\text{GF}(2^{128})$ in each round of the permutation P based on the following reasons. First of all, we expect some complicated interaction in different directions of the internal state S after the composition of ρ and ζ , which will active more 8-bit Sboxes in expectation and proved in the practical difficulty of solving the precise bitwise MILP or SAT models describing the differential propagation in Garnet, given the maximal freedom of injecting the difference in the state S . It is observed in the experiments that given a powerful computer server with 2 TB RAM and some large number of threads, the established state-of-the-art MILP/SAT solvers failed to give the answer of the corresponding MILP and SAT model after several weeks, even for 2 rounds of the permutation P in Garnet. The lower and upper bounds output by the solvers as intermediate results give some clues that the real number of the differentially active Sboxes for P^2 is not small, presumably close to the bounds given in Section 4 under the reasonable assumptions. This fact gives some justification that the linear transforms ρ and ζ exploited in Garnet contribute positively to the cryptographic strength of the whole algorithm.

Another reason for the selection of ρ and ζ is from the implementation aspect. The current definitions of the two finite fields $\text{GF}(2^{128})$ facilitates some lightweight MDS matrices in Garnet, which is verified by the efficient software/hardware implementation done so far.

Table 5. The Security statements of Garnet family of hash functions

Algorithm	Collision	Preimage	2nd Preimage
Garnet-512	256	512	512
Garnet-768	384	768	768
Garnet-1024a	512	1024	1024

4 Security Statements and Analysis

The classical security goals of the Garnet family of hash functions are given in Table 5. In general, we expect that Garnet with n -bit message digest will provide the $\frac{n}{2}$ -bit security against the collision attack, n -bit security against the

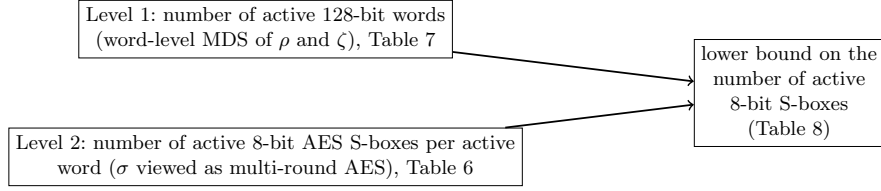


Fig. 4. The two-level combinatorial approach used to lower-bound the number of differentially active 8-bit S-boxes: the word-level activity count (from the MDS layers ρ, ζ) is combined with the byte-level active-S-box count inside each active word (from σ) to obtain the bounds of Tables 8.

pre-image and the 2nd pre-image attacks in the classical model. There is no quantum attacks on Garnet with a complexity lower than the generic quantum attacks in the quantum query model, as depicted later in this Section.

4.1 Security Analysis

Differential cryptanalysis was originally developed to attack block ciphers [4]. Differential attack was found to be important in analyzing the collision resistance of hash functions, and it has been applied to break MD4, MD5, SHA-0 and SHA-1 [6,11,35,36,37,38]. Thus, we first analyze the lower bound of the differential active S-boxes in the Garnet family of hash functions.

Due to the complex interaction of the linear transforms in the σ , ρ and ζ functions in the permutation P , it is challenging to determine the exact number of the differentially active 8-bit AES Sboxes for each round number. Instead, we adopt the following two-level combinatorial approach to compute the lower bound of the differentially active S-boxes in Garnet. Precisely, we take the following three-step procedure to determine this lower bound.

1. Compute the number of the differentially active 128-bit words with the increasing of the round number N_r .
2. Determine the number of the differentially active 8-bit AES Sboxes synchronized with the round number N_r .
3. Compute the final lower bound of the differentially active 8-bit AES Sboxes from the information obtained from the above two steps.

This two-level counting method is summarised in Fig. 4.

We have computed the lower bound of the differentially active 8-bit S-boxes under the following two assumptions.

1. A 1-bit difference can be introduced at any bit position of the internal state S , i.e., we do not consider the absorbing patterns in section 2.6
2. we simplify the interaction introduced by the ζ transform in the d diagonal directions by only considering the local σ transform at each 128-bit word as a multi-round encryption of the AES.

Under the above two assumptions, we have computed the lower bound of the differentially active 8-bit AES Sboxes of Garnet variants with the 4×4 state matrix, as shown in Table 8. The results in Table 8 is obtained by combining the underlying results in Tables 6 and 7, which are achieved by the Mixed Integer Linear Programming (MILP) method [27] and the mathematical induction according to the MILP results and the structure analysis of the AES-128 algorithm. Taking into account the maximal differential probability 2^{-6} of the 8-bit AES

Table 6. The minimum number of differentially active 8-bit S-boxes for N_r rounds of AES

N_r	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
min	1	5	9	25	26	30	34	50	51	55	59	75	76	80	84	100

Table 7. The minimum number of differentially active 128-bit words for N_r -round permutation in Garnet with the 4×4 state

N_r	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
min	1	8	12	16	20	24	28	32	36	40	44	48	52	56	60	64

Table 8. The lower bounds of differentially active 8-bit S-boxes for N_r -round permutation in Garnet with the 4×4 state

N_r	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
min	1	35	36	100	104	120	136	200	204	220	236	300	304	320	336	400

Sbox, Garnet has a comfortable security margin against differential attacks. For instance, any single differential characteristic over the 10 rounds applied in the absorbing phase of Garnet-512 has probability at most $(2^{-6})^{220} = 2^{-1320}$, which is far below the collision bound 2^{-256} ; hence no differential collision better than the birthday bound exists for any non-trivial number of rounds.

4.2 Linear Cryptanalysis

Linear cryptanalysis [25] approximates the permutation by linear relations $\langle \alpha, x \rangle \oplus \langle \beta, P(x) \rangle$ between an input mask α and an output mask β ; the data complexity to exploit a trail of absolute correlation c is $\mathcal{O}(c^{-2})$. For a layer of S-boxes the absolute correlation is the product of the per-S-box correlations, and the 8-bit AES S-box has maximal absolute correlation 2^{-3} .

A linear mask propagates through a linear map L as $(L^{-1})^\top$. Since both ρ and ζ are MDS transforms over $\text{GF}(2^{128})$, and the dual of an MDS code is again

MDS with the same branch number, the minimal number of *linearly* active S-boxes over N_r rounds equals the number of *differentially* active S-boxes given in Tables 8. We have confirmed this duality by re-running the MILP model with the transposed inverse matrices. Consequently, the best N_r -round linear trail has absolute correlation at most $(2^{-3})^{\#AS}$, requiring data complexity about $2^{6 \cdot \#AS}$. For Garnet-512 with $N_r = 10$, this gives a correlation of at most $(2^{-3})^{220} = 2^{-660}$ and a data complexity of at least 2^{1320} . Therefore, Garnet enjoys the same comfortable security margin against linear cryptanalysis as against differential cryptanalysis.

4.3 Algebraic Degree, Zero-Sum and Integral Distinguishers

We now bound the algebraic degree of the permutation P , which controls higher-order differential, zero-sum and integral distinguishers. A crucial structural property of Garnet is that *each round contains a single non-linear layer σ (SubWord)*, whose only source of non-linearity is the 8-bit AES S-box of algebraic degree 7; the layers ρ , θ and ζ are affine. Hence each output bit of one round has algebraic degree at most 7 in the input bits, and over N_r rounds

$$\deg(P^{N_r}) \leq 7^{N_r}, \quad \deg(P^{-N_r}) \leq 7^{N_r}. \quad (3)$$

This trivial bound is informative only while $7^{N_r} < n$, i.e. $N_r \leq 3$ for the $n = 2048$ -bit state.

For larger N_r we use the bound of Boura, Canteaut and De Cannière [9]. For a layer F of S-boxes over $\text{GF}(2^{n_0})$ and any function G ,

$$\deg(F \circ G) \leq n - \frac{n - \deg(G)}{\gamma}, \quad \gamma = \max_{1 \leq i \leq n_0-1} \frac{n_0 - i}{n_0 - \delta_i}, \quad (4)$$

where δ_i is the maximal algebraic degree of a product of any i output coordinates of the S-box. Equivalently, writing the degree deficiency $\overline{\deg} = n - \deg$, each S-box layer shrinks it by at most a factor γ . Computing the δ_i from the algebraic normal form of the AES S-box (and of its inverse) yields $\delta_i = (7, 7, 7, 7, 7, 7, 7, 8)$ for $i = 1, \dots, 8$: a product of any ≤ 7 output coordinates of $A \circ x^{-1}$ never reaches the top monomial, so $\gamma = \max_{1 \leq i \leq 7} (8 - i)/(8 - 7) = 7$. Combining (3) and (4) with $\deg(P^1) = 7$ and $n_0 = 8$, $n = 2048$ gives the provable upper bounds in Table 9.

Table 9. Provable upper bounds on the algebraic degree of N_r rounds of the Garnet permutation P (4×4 state, $n = 2048$).

N_r	1	2	3	4	5	6	7
$\deg(P^{N_r}) \leq$	7	49	343	1804	2013	2043	2047

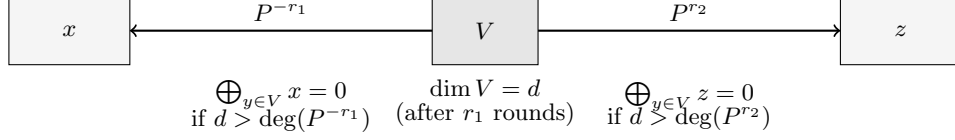


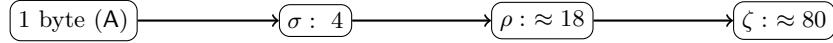
Fig. 5. The inside-out zero-sum construction. A d -dimensional affine subspace V is placed in the middle (after r_1 rounds); computing $x = P^{-r_1}(y)$ backward and $z = P^{r_2}(y)$ forward for all $y \in V$ yields a zero-sum of the (r_1+r_2) -round permutation whenever $d > \max(\deg(P^{-r_1}), \deg(P^{r_2}))$.

Zero-sum distinguishers. A zero-sum for P is a set Z with $\bigoplus_{x \in Z} x = 0$ and $\bigoplus_{x \in Z} P(x) = 0$; a random permutation of $\text{GF}(2)^n$ has degree $n-1$ and does not cheaply possess such structured sets, so exhibiting one is a distinguisher [1,9]. Using the inside-out construction, we choose a d -dimensional affine subspace V in the middle, after r_1 rounds; for every $y \in V$ we compute $x = P^{-r_1}(y)$ and $z = P^{r_2}(y)$. By the higher-order-derivative property [21], $\bigoplus_{y \in V} x = 0$ when $d > \deg(P^{-r_1})$ and $\bigoplus_{y \in V} z = 0$ when $d > \deg(P^{r_2})$, so $\{x \mapsto z\}$ is a zero-sum for $P^{r_1+r_2}$ provided $d = \max(\deg(P^{r_1}), \deg(P^{r_2})) + 1 \leq n-1$ (the construction is illustrated in Fig. 5). The time complexity is 2^d evaluations of $P^{r_1+r_2}$ with negligible memory. The balanced split $r_1 = r_2 = 4$ gives the deepest distinguisher, $d = \deg(P^4) + 1 = 1805$, i.e. a zero-sum over 8 rounds with complexity 2^{1805} . Smaller splits give the ladder $2/2^8$, $4/2^{50}$, $6/2^{344}$, $8/2^{1805}$. We stress that this is a structural property of the reduced-round permutation only: its complexity 2^{1805} far exceeds all security claims of Table 5, and any externally observable transformation in Garnet applies at least $14 + 10 = 24$ rounds (Table 3); thus the zero-sum yields no attack on the hash functions.

Integral distinguishers. As a low-data special case, a single active byte (one byte ranging over all 256 values, the rest of the state fixed) yields a 2-round integral distinguisher: the 2^8 outputs sum to zero on the whole state. We verified this exactly on the reference implementation and confirmed (over many random constant parts, to exclude false positives) that no output bit remains balanced after 3 rounds, in agreement with $\deg(P^3) \leq 343$ (a forward 3-round zero-sum would need a 344-dimensional cube). The byte-level propagation behind this property is illustrated in Fig. 6. The integral distinguisher therefore reaches exactly 2 rounds with 2^8 data, well below the round numbers used in any phase.

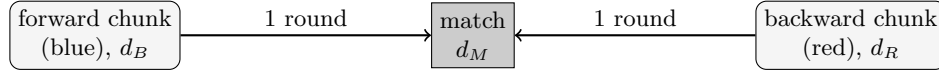
4.4 Meet-in-the-Middle Preimage Attack

Meet-in-the-middle (MITM) preimage attacks are an important technique in the analysis of hash functions. Qin et al. applied this technique to preimage attacks on SHA-3 [31], and subsequently, Dong et al. further improved it and proposed a MITM preimage attack applicable to sponge-based hash functions [16]. For the preimage attack on Garnet, we adopt the generic MITM preimage attack



after one round: 80/256 bytes active, 176/256 still balanced (B)

Fig. 6. Byte-level propagation of one active byte (an integral A -set) through one Garnet round. The within-word **MixColumns** inside σ turns one active byte into four; ρ and ζ then spread the activity, yet 176 of the 256 bytes remain balanced after one round, and the whole state is balanced after two rounds.



one-round full diffusion of $\rho, \zeta \Rightarrow$ each chunk spans at most 1 round;
for ≥ 2 rounds the blue and red influences overlap, leaving no neutral words

Fig. 7. The meet-in-the-middle configuration on Garnet. The forward (blue) and backward (red) chunks meet at a matching point of dimension d_M . Only the $1 + 1$ split is feasible ($d_B = d_R = 72$, $d_M = 24$ bytes); it cannot be extended because ρ and ζ diffuse fully within one round.

framework of Dong et al. [16], in which the state is coloured forward (blue) and backward (red); one searches, by MILP, for neutral degrees of freedom d_B, d_R that do not affect the opposite chunk together with a matching point of dimension d_M , maximising $\min(d_B, d_R, d_M)$. The framework gives a preimage cost of $2^{n - \min(d_B, d_R, d_M)}$ and a collision cost of $2^{n/2 + d_M/2 - \min(d_B, d_R, d_M)}$, where d_B, d_R, d_M are neutral degrees of freedom (not the capacity size).

Running this search on Garnet shows that a valid configuration exists only when each chunk spans a single round (a $1 + 1$ split), with at best $d_B = d_R = 72$ bytes of neutral freedom and a matching point of $d_M = 24$ bytes (see Fig. 7). The reason is the double MDS diffusion: the layers ρ and ζ together achieve full diffusion within one round (any single active word influences all 16 words after one round), so for ≥ 2 rounds in either direction the forward and backward influences overlap completely and no neutral words survive. Because the configuration cannot be extended to span the ≥ 10 rounds applied between message injection and the squeezed output, the framework yields no preimage (or collision) attack on Garnet faster than the generic bounds of Table 5.

4.5 Further Meet-in-the-Middle Analysis

Let us consider some further Meet-in-the-Middle cryptanalysis of Garnet.

Scope of the MITM model. Meet-in-the-middle (MITM) attacks are a major tool for preimage, pseudo-preimage, second-preimage and partial-collision cryptanalysis of hash functions. Modern MITM attacks are not limited to a naive split into

two naturally independent chunks. They may use splice-and-cut, initial structures, partial or indirect matching, message-state freedom, superposition-state propagation, guess-and-determine, sieve-in-the-middle, and, for sponge constructions, constrained-input constrained-output (CICO) and partial-target-preimage subproblems. We therefore distinguish between a first-order internal-permutation dependency search and a full-mode Sponge/Sponge-DM conversion analysis.

Basic dependency-coloring search. As a first screening test, we apply a blue/red dependency-coloring model to the Garnet permutation. The model searches for forward and backward neutral degrees of freedom d_B, d_R , together with a matching dimension d_M , leading to the standard estimate

$$T_{\text{pre}} \approx 2^{n - \min(d_B, d_R, d_M)}.$$

In this basic model, where initial-structure cancellation, superposition-state propagation, indirect partial matching and full Sponge-mode conversion are not included, the best configuration found is a 1 + 1-round independent-chunk split with $d_B = d_R = 72$ bytes and $d_M = 24$ bytes. This configuration cannot cover the full number of rounds used in any absorbing, finalization or squeezing phase of Garnet, and it does not yield a preimage or collision attack faster than the generic bounds.

Limitations of the current search. The preceding result should not be read as a proof excluding all MITM attacks. The strong one-round diffusion of the ρ and ζ layers destroys simple independent chunks, but recent automated analyses of AES-like hashing and sponge constructions show that an attacker may impose initial-structure constraints so that the influence of one neutral set on the opposite chunk becomes constant or is cancelled. Mixed red/blue cells may also remain useful when their dependencies are linearly superposed through public linear layers, and a matching point may use only a projection or an indirect linear relation. Hence one-round full diffusion is an important design argument, but not a formal impossibility certificate for all modern MITM variants.

Full Sponge-mode conversion. For the Sponge-mode instances Garnet-512 and Garnet-768, a full attack must connect to the fixed initialization, valid padding, rate-word message injection, length injection, cross-call state connection and digest projection. For preimage analysis, the squeezing side naturally introduces a CICO problem. For collision analysis, one should distinguish direct digest partial-target collisions from inner-state collisions. Since the standard Garnet instances have $n < c$, the direct digest partial-target framework is the primary benchmark for ordinary digest collisions, while an inner-state collision is a stronger and usually harder target.

Sponge-DM closure. For Garnet-1024a, the absorbing update is

$$X_j = S^j \oplus I_r(M_j), \quad S^{j+1} = P^b(X_j) \oplus X_j.$$

Consequently, a two-branch MITM or differential path that merges after one absorbed block has to satisfy the full-state closure equation

$$P^b(X) \oplus P^b(X') = X \oplus X', \quad \Delta X \in \text{Im}(I_r).$$

This Davies–Meyer feed-forward equation must be treated as an explicit matching or closure constraint in a MITM model.

The Current-Stage Conclusion. No MITM preimage, second-preimage or collision attack faster than the generic bounds is obtained by the present search. However, the current result is a negative result only for the stated basic dependency-coloring model. A further in-depth assessment should provide a reproducible MILP/SAT model and search certificates covering initial structures, partial or indirect matching, superposition states, guess-and-determine, sieve-in-the-middle, CICO constraints, digest projection and the Sponge-DM feed-forward closure.

4.6 Other Cryptanalysis

We have further evaluated Garnet against the following techniques; in each case the one-round full diffusion of ρ and ζ keeps the reachable number of rounds very small.

Rebound attacks. [26] Since ρ (`MixColumn`) preserves the column structure, the map $\sigma \circ \rho \circ \sigma$ acts independently on each column and forms a 512-bit “Super-S-box” with 2^{512} degrees of freedom. However, ζ (`MixDiagonal`) breaks the column alignment in every round, so no clean two-round Super-S-box occurs in the round sequence; the inbound phase covers at most about two S-box layers and the outbound probability collapses under full diffusion. An inbound of about two rounds plus one outbound round gives a reachable depth of about three rounds (from four rounds onward the outbound activates ≥ 33 S-boxes, i.e. probability $\leq 2^{-198}$), far below the round numbers in any phase.

Boomerang and rectangle attacks. [34] Splitting $P = E_1 \circ E_0$, each half keeps a difference sparse for at most one round before ρ and ζ make it fully active, so the two characteristics have probabilities $p, q \leq 2^{-6 \cdot \#AS}$ that collapse quickly; the boomerang probability $p^2 q^2$ drops below 2^{-n} within about three rounds.

Impossible differential and zero-correlation. [7,8] A truncated difference (or mask) becomes fully active after one round in both directions, so the miss-in-the-middle contradiction window is about one round and these distinguishers reach only about two rounds. Moreover, on the 2048-bit state the data needed to exploit such a truncated property approaches the full codebook (for example, confirming the 1-round impossible event “one active word $\rightarrow$ one active word” requires about 2^{1916} data), so they are of no practical use. By the equivalence between zero-correlation linear hulls and integrals, the two-round zero-correlation property coincides with the two-round integral above.

Interpolation attacks. [22] The only non-linearity is the AES S-box $\mathcal{A} \circ x^{-1}$, whose univariate degree over $\text{GF}(2^8)$ is the maximal 254. After one to two rounds

the output, viewed as a multivariate polynomial in the input bytes, already has more than 2^{128} distinct monomials, so reconstruction is infeasible; the large-field MDS layers do not lower the degree.

Invariant subspace and slide attacks. [24] The MDS matrices T_1 and T_3 satisfy $T^4 = I$ over their respective fields, but the distinct field moduli used by ρ and ζ , the round-dependent constants added by θ , and the non-linear layer σ prevent this from lifting to any invariant subspace of P (the bit-level combined matrix of $\zeta \circ \rho$ has full rank with no shared eigenvectors). The round-dependent Fibonacci constants of θ , chosen to destroy the symmetry between the columns, also rule out slide attacks.

Rotational cryptanalysis. [19] Garnet is not an ARX design, and the AES S-box is not rotation-equivariant, so a rotational pair is destroyed after a single σ layer; the technique does not apply.

Quantum cryptanalysis [17,10]. We find no dedicated quantum procedure that exploits the concrete structure of Garnet, so the best quantum attacks remain the generic ones. The query model, the generic attack bounds, the capacity-mediated $2^{c/3}$ inner-state bound and the per-instance exponents are detailed in Section 4.7.

Pseudo-randomness test [30]. The digest streams of all three Garnet instances pass the full NIST SP 800-22 suite (188 sub-tests) under five message-sample ensembles at the high-confidence 1000×10^6 -bit setting; details are in Appendix C.

Length-extension attacks. Garnet is built from the Sponge/Sponge-DM mode shown in Figs.1 and 2 and is not a Merkle–Damgård hash. It never outputs its internal chaining state: the capacity is hidden, so a published digest yields no continuable state, thus is immune to the length-extension attacks. The following Section 4.8 presents the details.

Table 10. Best reduced-round distinguishers found on the Garnet permutation P (Garnet-512).

Technique	Rounds on P	Complexity
Zero-sum / higher-order	8	2^{1805}
Integral / division property	2	2^8
Differential (collision)	—	no gain over birthday
Linear / zero-correlation	~ 2	$\geq 2^{1320} / 2^{2048}$
Boomerang / rectangle	~ 3	—
Impossible differential	~ 2	$\approx 2^{1916}$
Meet-in-the-middle	2	no gain over generic
Rebound	~ 3	—
Interpolation	~ 1	infeasible
Invariant subspace / slide / rotational	—	not applicable

Table 10 summarises the deepest reduced-round distinguisher we found on the permutation P for each technique. The strongest is an 8-round zero-sum with complexity 2^{1805} ; all the difference- and mask-based techniques stop within about four rounds because the double MDS layer $\rho + \zeta$ diffuses fully in a single round, and the algebraic-degree based techniques are limited by the fast degree growth $\deg(P^{N_r}) \leq 7^{N_r}$. Since every phase of every Garnet variant applies at least 10 rounds (Table 3) and the above complexities far exceed the security claims, none of these analyses threatens the full-round hash functions.

4.7 Quantum Security Analysis

We analyse the security of the Garnet family in the quantum query setting.

For an n -bit digest, the best known generic quantum attacks are determined primarily by the output length. Grover search gives a generic preimage and second-preimage cost of $\Theta(2^{n/2})$ quantum queries for a random n -bit function [17]. The Brassard–Hoyer–Tapp algorithm finds a collision in $O(2^{n/3})$ quantum queries when the required quantum memory and qRAM-style access are available [10]; under a low-quantum-memory model, the best known generic exponent is $2n/5$ [12]. These are upper bounds arising from generic attacks.

Table 11. Generic quantum-attack complexity exponents for Garnet. The entries are base-2 logarithms of query counts and are determined by the digest length n .

Algorithm	n	BHT collision	Low-memory collision	Grover preimage
Garnet-512	512	170.7	204.8	256
Garnet-768	768	256.0	307.2	384
Garnet-1024a	1024	341.3	409.6	512

The capacity $c = b - r$ also constrains attacks that try to recover or collide internal state information rather than attack only the public digest. In the ideal-permutation analysis of sponge-like constructions, the relevant generic quantum scale for capacity-mediated inner-state attacks is governed by approximately $2^{c/3}$ queries. For Garnet-512 and Garnet-768, one has $c/3 > n/2$ at every supported rate; hence the Grover bound on the public n -bit output is the binding generic preimage constraint. For Garnet-1024a, $c = 1152$ gives $c/3 = 384 < 512 = n/2$. Its 512-bit quantum preimage and second-preimage target should therefore be read as a design target conditioned on the full-state Sponge-DM feed-forward and on the absence of structural quantum shortcuts in the concrete permutation, rather than as a consequence of capacity alone.

Such dedicated quantum rebound and meet-in-the-middle results target *AES-like hashing in compression-function modes*—a block cipher or permutation used in Davies–Meyer, Matyas–Meyer–Oseas or Miyaguchi–Preneel form inside a (wide-pipe) Merkle–Damgård iteration, as in Whirlpool, AES-MMO and Grøstl—where the chaining value and its feed-forward are exposed and are exactly the

structure the attacks exploit [15,32]. Garnet instead uses the Sponge and Sponge-DM modes, which release only an n -bit projection of the state and keep the capacity hidden, so these compression-function attacks do not transfer directly; the analyses that do apply, e.g., the sponge-specific meet-in-the-middle frameworks [16] and rebound applied to the public permutation, reach only a few rounds against the full construction, and their quantum forms additionally require a separate model such as quantum-accessible quantum memory (QRAQM). For Sponge-DM the per-block Davies–Meyer feed-forward must be modelled as an extra matching surface (Section 4.8 and the meet-in-the-middle analysis above), while the hidden capacity is retained. In all cases the reachable depth stays far below the 10–16 rounds applied in each Garnet phase, so no advantage over the generic quantum exponents of Table 11 is obtained.

4.8 Length-Extension Attacks

Garnet is not a Merkle–Damgård construction that outputs its internal chaining value. In the Garnet instances based on Sponge mode, after the final padded message block is absorbed, the original bit-length $mlength$ is injected, the middle permutation is applied, a further pre-squeezing permutation is applied, and only selected output words are released. The hidden capacity part is never directly output. Consequently an adversary given only a digest does not obtain a valid internal state from which the processing of an appended suffix can be continued. The one-bit padding followed by zero padding is injective at the 128-bit-block level, and the separate $mlength$ injection binds the digest to the original message length. These facts rule out the direct Merkle–Damgård-style length-extension attack under the ideal-permutation assumption.

For the Garnet instances built on the Sponge-DM mode, the same conclusion depends on the full-state Davies–Meyer feed-forward after every absorbed block. The update $S^{j+1} = P^b(X_j) \oplus X_j$ hides the internal state behind a permutation-and-feed-forward transformation, and the final length injection and output projection again prevent an attacker from recovering a continuable chaining state from the digest alone.

With the parameters in Table 3, the capacities are at least 1024 bits for Garnet-512, 1536 bits for Garnet-768, and 1152 bits for Garnet-1024a; thus the corresponding $c/2$ margins are 512, 768, and 576 bits, respectively, before considering digest-length limitations.

5 Performance Evaluation

Garnet can be implemented efficiently on a wide range of platforms: from hardware to 128-bit/256-bit instructions. The reason is that the generalized AES structure allows Garnet being constructed from relatively simple elements. Each transform of the one round permutation $P = \zeta \circ \theta \circ \rho \circ \sigma$ can be executed in parallel, if we regard the `AddConstant` operation θ as xoring 0s to the non-specified positions of the state S . This parallel feature of Garnet allows it to

run efficiently on modern microprocessors. For the 4×4 states, the two MDS matrices defined over two different $\text{GF}(2^{128})$ fields are very lightweight, and thus efficient in both hardware and software.

5.1 Software

The software implementation of Garnet is run on the Intel(R) Xeon(R) Gold 6242R, Intel(R) Core(TM) Ultra 5 and (USTC). The operating systems are 64-bit Windows 11 and Ubuntu 20.04. The compilers being used are Microsoft Visual Studio C++ compiler, Intel C++ compiler 2025, gcc/g++ and Clang. The hash speed is measured by hashing a 32768-byte buffer for 10^7 times and using both 'clock()' and 'rdtsc' in C/C++ to measure the hash computation. With the new AES-NI instruction, the speed of Garnet-512 with $r = 1024$ -bit (non-optimized) is about 8.87 cycles/byte on the Intel(R) Xeon(R) Gold 6242R CPU running 64-bit operating system with Microsoft Visual Studio C++ compiler, while the speed of the same Garnet instance is about 6.3876 cycles/byte after using macros and inline functions with the Intel C++ compiler 2025. It is worth noting that the same code (using macros and inline functions) of Garnet-512 with the rate $r = 1024$ -bit running on the Intel(R) Core(TM) Ultra 5 125H of 1.20GHZ is about 2.69 to 3.15 cycles/byte with 64-bit windows 11 operating system. For the linux platform Ubuntu 24.04, we get that the hash speed is around 7.00 to 8.00 cycles/byte for the Garnet-512 instance with the rate $r = 1024$ -bit. We also measure the hash speed of the Garnet algorithm on the ngcc_bench linux

Table 12. The Hash speed of the Garnet algorithm on the Intel(R) Core(TM) Ultra 5 125H of 1.20 GHZ

Algorithm	Size(bytes)	Garnet-512					Garnet-768	Garnet-1024a
rate, bits		512	640	768	896	1024	512	896
Speed (cycles/byte)	64	25.63	18.66	18.73	18.86	18.78	27.63	23.41
Speed (cycles/byte)	1024	6.71	5.47	4.68	4.34	3.98	7.38	5.23
Speed (cycles/byte)	4096	6.52	5.43	3.88	3.49	3.07	6.73	4.17
Speed (cycles/byte)	16384	6.03	4.43	3.70	3.23	2.94	6.36	4.15
Speed (cycles/byte)	32768	5.92	4.57	3.95	3.23	2.83	6.30	4.14

x86_64 platform provided at the website https://niccs.org.cn/symbzyjy/tzgg/pc/content/2066796358054449152/content_2066796358054449152.html. The result is as follows. For the same Garnet-512 instance with the rate $r = 1024$ -bit, we use the following compilation settings:

```
gcc -O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame
-pointer -std=c99 -Wpedantic -Wall -Wextra main.c
Garnet_512.c -o test_hash
```

When the input buffer is 4096-byte, we have obtained that the throughput is 1033.94 Mbps on a CPU of 0.95 GHZ with the memory profiling result (x86) of text size 4322 bytes and data size 4896 bytes. When the input size is increasing from 4096-byte to 32768-byte, the throughput is increasing to around 1500 Mbps. In our experiments, we even get the hash speed of 2012.67 Mbps when the input message is of 327680-byte.

Table 13. The Hash speed of the Garnet algorithm on the ngcc.bench virtual machine with x86_64 linux

Alg.	Size(bytes)	Garnet-512					Garnet-768	Garnet-1024a
		512	640	768	896	1024	512	896
Thpt. (Mbps)	64	116.34	118.93	122.76	128.95	137.46	103.7	120.63
Thpt. (Mbps)	1024	239.17	376.81	398.26	468.29	478.98	367.68	303.24
Thpt. (Mbps)	4096	579.24	724.13	778.79	853.72	878.46	452.73	308.27
Thpt. (Mbps)	32768	978.29	1077.29	1182.67	1207.19	1301.09	559.47	446.37
Thpt. (Mbps)	327680	1178.67	1228.93	1383.36	1430.29	1587.09	624.9	548.84

5.2 Embedded Systems

The performance of the Garnet family of hash functions on the embedded system is an important issue, since most smart cards have such a processor and many cryptographic applications run on smart cards. Recall that the one round permutation $P = \zeta \circ \theta \circ \rho \circ \sigma$, where the transform σ is one-round AES encryption [13] with the multiplication over $\text{GF}(2^8)$. It is well known that the multiplication over $\text{GF}(2^8)$ can be implemented as a small table look-up and the sub-bytes operation in σ is realized using the composition field arithmetics. The different steps of the σ operation can be combined in a single set of look-up tables, thus allowing for very fast implementations on processors on the 32-bit embedded systems, as stated in [13]. It is worth noting that the usual way of storing the four tables which has 256 4-byte word entries each will require 4 KB of storage, but the look-up table implementation can be realized with only one table, i.e., with a total table size of 1 KB due to the entries of the former four tables are rotated versions of one another.

Both the `MixColumn` ρ transform and the `MixDiagonal` ζ transform can be efficiently implemented on the 32-bit embedded systems by appropriately combining the union structure to define a 128-bit unit. The multiplication with the primitive element α over the finite field $\text{GF}(2^{128})$ defined by the primitive polynomial $x^{128} + x^7 + x^2 + x + 1$ can be implemented efficiently through a table look-up as

```
static const uint64_t reduction_table_1_MC4[2] = {0x0, 0x87,}
```

depending on the most significant bit of the argument. Similarly, the multiplication with the primitive element α over the finite field $\text{GF}(2^{128})$ defined by the primitive polynomial $x^{128} + x^{29} + x^{15} + x^2 + 1$ can be implemented efficiently through a table look-up as

```
static const uint64_t reduction_table_1_MD4[2] = { 0x0, 0
x0000000020008005,}
```

depending on the most significant bit of the argument. These are the cases for the 4×4 matrix state S . Please refer to the implementation codes for the details. As a result of all the above techniques, the 32-bit embedded system implementation of Garnet-512 with the rate $r = 1024$ -bit on ARM-v7m (CC-MARM on) (`stm32f407vgt6`) which is the same test platform as the NIST `pqm4` framework [20], with a CPU clock of 109 MHz is shown in Table 14. For the implementation in Table 14, the heap status is as follows: the current free heap is 8760 bytes and the minimum ever free is 4640 bytes. The compiler used in Table 14 is `arm-none-eabi-gcc 14.2.1` at the optimize level `-O3 -mthumb -mfloat-abi=hard` running the `FreeRTOS` operating system with the `Run-Time Stats` and `Stack Monitoring` on. It is worth noting that the typical ARM embedded platform does not support the AES-NI like hardware instructions, especially for the Cortex-M series. Thus, we do not take the acceleration of the AES-NI instructions into account when making the implementation in the 32-bit embedded systems. From Tables 14 and 15, we can see that Garnet-512/ $r=1024$ is comparable to or slightly better than the industrial standard SHA3-512 on the 32-bit embedded system without the acceleration of some special bitwise instructions.

We have further implemented the Garnet family of hash functions on the 32-bit `Espressif ESP32-C6`/RISC-V embedded system running the `Zephyr RTOS v4.4.0` operating system with a CPU clock of 160 MHz as shown in Tables 16 and 17. We adopt the configuration of `Internal SRAM Accelerated` with the `-O3` optimization and use the `Espressif SOC HAL (True CPU Cycles)` as the cycle counter. From Tables 16 and 17, we can see that Garnet-512/ $r=1024$ outperforms the industrial standard SHA3-512 on the 32-bit `Espressif ESP32-C6`/RISC-V embedded system by a factor of about 10% and the Garnet-1024a instance have a very robust performance for the long messages with the 1024-bit security.

5.3 Hardware

The hardware implementation of Garnet is simple and efficient because of the generalized AES structure. Garnet exploits 2048-bit or 3200-bit memory for stor-

Table 14. The Comprehensive Benchmark implementation on ARM-v7m without the AES instructions

Variant	Size(Bytes)	Cycles	Time(μ s)	CPB	MiB/s
Garnet-512/r=512	64	164172	1496.4	2565.19	0.041
	1024	697927	6361.3	681.57	0.154
	4096	2406924	21938.1	587.63	0.178
	8192	4685719	42708.4	571.99	0.183
	16384	9243480	84250.5	564.18	0.185
Garnet-512/r=640	64	128668	1172.8	2010.44	0.052
	1024	556198	5069.5	543.16	0.193
	4096	1946141	17738.3	475.13	0.220
	8192	3764202	34309.1	459.50	0.228
	16384	7400169	67449.5	451.67	0.232
Garnet-512/r=768	64	425116	3874.76	6642.44	0.02
	1024	935550	8527.15	1827.25	0.06
	4096	4506931	41078.80	1100.32	0.10
	8192	1336665	12183.14	326.33	0.32
	16384	1336665	12183.14	326.33	0.32
Garnet-512/r=896	64	128683	1172.9	2010.67	0.052
	1024	450421	4105.4	439.86	0.238
	4096	1416149	12907.6	345.74	0.303
	8192	2739606	24970.4	334.42	0.313
	16384	5350945	48771.6	326.60	0.320
Garnet-512/r=1024	64	128719	1173.2	2011.23	0.052
	1024	414564	3778.6	404.85	0.258
	4096	1272580	11599.0	310.69	0.337
	8192	2417019	22030.1	295.05	0.355
	16384	4706094	42894.1	287.24	0.364
Garnet-768	64	178850	1630.1	2794.53	0.037
	1024	765499	6977.2	747.56	0.140
	4096	2643860	24097.7	645.47	0.162
	8192	5148309	46924.7	628.46	0.166
	16384	10157321	92579.7	619.95	0.169
Garnet-1024a	64	151909	1384.6	2373.58	0.044
	1024	544002	4958.4	531.25	0.197
	4096	1721107	15687.2	420.19	0.249
	8192	3334156	30389.4	407.00	0.257
	16384	6516836	59398.2	397.76	0.263

- CPU Clock: 109 MHz — Optimization: ARM-v7m (CCMRAM On)
- Stack High Water Mark: 512 words remaining

Table 15. Comparison with the wolfSSL SHA3-512 on the same platform ARM Cortex-M4F (stm32f407vgt6)

Sizes(16 KB)	Garnet-512/r=1024	SHA3-512 (wolfSSL)
CPB	287.24	290.11
MiB/s	0.364	0.360

ing the state of the permutation P with some extra memory to store the round constants. We first look at the hardware implementation of AES-128, which is the basis of the hardware implementation of the Garnet hash family. There are quite some research on the compact hardware implementation of AES-128. For example, a very compact hardware implementation of AES-128 requiring only 2400 GE is presented in [28]. Since the state is about 16 times that of AES-128, thus it is reasonable to estimate that Garnet will requires 16 times more gates than AES-128, that is $16 \times 2400 \doteq 38K$ GE. The two MDS matrix over $GF(2^{128})$ selected in the Garnet family of hash functions is especially suitable for hardware implementation. This property preserves for all the different MDS matrices of 4×4 in Garnet, which will greatly reduce the number of gates in hardware implementation. Further, it is worth noting that there is some tradeoffs for the hardware implementation of Garnet. It is expected that the future optimized hardware implementations of Garnet would be developed by fully exploiting the structure feature and potential of the algorithm.

6 Feature Statements

Garnet hash algorithms have the following features and advantages:

1. Simple design. The overall design is very simple due to the simple permutation constructed based on the generalized AES structure.
2. New insights. Though the overall structure of Garnet is very simple, we adopt several definitions of the finite field underlying the MDS matrices and try to open the byte boundaries in several different ways. The composition of several different bitwise transforms and the non-linear transform will increase the number of the byte-wise active S-boxes and thus the security, compared to that of the purely aligned design approach like in AES itself.
3. Garnet can be implemented efficiently in both hardware and software. The reason is that the generalized AES structure will allow Garnet being built relative easily. Garnet is efficient in hardware since simple components and identical round functions are used.
4. It is convenient to extend Garnet to meet several other requirements, such as the tweakable information being put into the large internal state. Garnet can be used directly to construct a message authentication code (MAC), the extendable output functions (XoF) and stream ciphers.

Table 16. The Comprehensive Benchmark implementation on ESP32-C6 / RISC-V RV32IMAC

Variant	Size(Bytes)	Cycles	Time(μ s)	CPB	MiB/s
Garnet-512/r=512	64	146278	914.2	2285.59	0.067
	1024	586874	3668.0	573.12	0.266
	4096	1997089	12481.8	487.57	0.313
	8192	3877231	24232.7	473.29	0.322
	16384	7637906	47736.9	466.18	0.327
Garnet-512/r=640	64	116982	731.1	1827.84	0.083
	1024	470621	2941.4	459.59	0.332
	4096	1619620	10122.6	395.42	0.386
	8192	3122429	19515.2	381.16	0.400
	16384	6128006	38300.0	374.02	0.408
Garnet-512/r=768	64	116972	731.1	1827.69	0.083
	1024	412408	2577.6	402.74	0.379
	4096	1357605	8485.0	331.45	0.460
	8192	2627707	16423.2	320.77	0.476
	16384	5138749	32117.2	313.64	0.486
Garnet-512/r=896	64	117051	731.6	1828.92	0.083
	1024	383717	2398.2	374.72	0.407
	4096	1184412	7402.6	289.16	0.528
	8192	2281376	14258.6	278.49	0.548
	16384	4446291	27789.3	271.38	0.562
Garnet-512/r=1024	64	117032	731.5	1828.62	0.083
	1024	354310	2214.4	346.01	0.441
	4096	1067125	6669.5	260.53	0.586
	8192	2017382	12608.6	246.26	0.620
	16384	3917797	24486.2	239.12	0.638
Garnet-768	64	159431	996.4	2491.11	0.061
	1024	643525	4022.0	628.44	0.243
	4096	2192508	13703.2	535.28	0.285
	8192	4258480	26615.5	519.83	0.294
	16384	8390088	52438.1	512.09	0.298
Garnet-1024a	64	138200	863.8	2159.38	0.071
	1024	462505	2890.7	451.67	0.338
	4096	1435248	8970.3	350.40	0.435
	8192	2768273	17301.7	337.92	0.452
	16384	5398465	33740.4	329.50	0.463

- CPU Clock: 160 MHz — Optimization: GCC (Zephyr SDK) -O3
- RAM optimization: T-Table in Internal SRAM

Table 17. Comparison with the wolfSSL SHA3-512 on the same platform RISC-V RV32IMAC

SIZES(16 KB) Garnet-512/r=1024 SHA3-512 (wolfSSL)		
CPB	239.16	263.04

7 Conclusions

The Garnet family of hash functions is proposed in this paper. There are three algorithms involved in the Garnet family, i.e., Garnet-512, Garnet-768 and Garnet-1024a. These hash functions are expected to be implemented efficiently on a wide range of platforms including hardware and software with a comfortable security margin according to the digest size in both the classical and quantum settings. Any third-party cryptanalysis is welcome.

References

1. Aumasson, J.-P., Meier, W., Zero-sum distinguishers for reduced Keccak- f and for the core functions of Luffa and Hamsi. Rump session of CHES 2009.
2. Bertoni, G., Daemen, J., Peeters, M., Assche, G.V., Sponge Functions. *ECRYPT Hash Workshop*, 2007.
3. Bellini, E., Gerault, D., Protopapa, M., et al., CLAASP: a Cryptographic Library for the Automated Analysis of Symmetric Primitives. Selected Areas in Cryptography–SAC 2023; software: <https://github.com/Crypto-TII/claasp>.
4. Biham, E., Shamir, A., Differential cryptanalysis of DES-like cryptosystems. *Advances in Cryptology–Crypto’1990 LNCS vol. 537*, pp.2–21, Springer-Verlag, 1991.
5. Biham, E., Chen, R., Near-collisions of SHA-0. *Advances in Cryptology–Crypto’2004 LNCS vol. 3152*, pp.290–305, Springer-Verlag, 2004.
6. Biham, E., Chen, R., Joux, A., Carribault, P., Lemuet, C., Jalby, W., Collisions of SHA-0 and Reduced SHA-1. *Advances in Cryptology–Eurocrypt’2005 LNCS vol. 3494*, pp.36–57, Springer-Verlag, 2005.
7. Biham, E., Biryukov, A., Shamir, A., Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. *Advances in Cryptology–EUROCRYPT 1999, LNCS vol. 1592*, pp. 12–23, Springer, 1999.
8. Bogdanov, A., Rijmen, V., Linear hulls with correlation zero and linear cryptanalysis of block ciphers. *Designs, Codes and Cryptography* 70(3), pp. 369–383, 2014.
9. Boura, C., Canteaut, A., De Cannière, C., Higher-order differential properties of Keccak and Luffa. *Fast Software Encryption–FSE 2011, LNCS vol. 6733*, pp. 252–269, Springer, 2011.
10. Brassard, G., Hoyer, P., Tapp, A., Quantum cryptanalysis of hash and claw-free functions. *Latin American Symposium on Theoretical Informatics–LATIN 1998, LNCS vol. 1380*, pp. 163–169, Springer, 2006.
11. Chabaud, F., Joux, A., Differential Collisions in SHA-0. *Advances in Cryptology–Crypto’1998 LNCS vol. 3494*, pp.56–71, Springer-Verlag, 1998.
12. Chailloux, A., Naya-Plasencia, M., Schrottenloher, A., An efficient quantum collision search algorithm and implications on symmetric cryptography. *ASIACRYPT 2017, LNCS vol. 10625*, pp. 211–240, Springer, 2017.

13. Daemen, J., Rijmen, V., The Design of Rijndael: AES - The Advanced Encryption Standard. Springer, 2020.
14. Dobbertin. H., Cryptanalysis of MD4. Journal of Cryptology vol. 11, pp.253–271, Springer-Verlag, 1998.
15. Dong, X., Zhang, Z., Sun, S., Wei, C., Wang, X., Hu, L., Automatic Classical and Quantum Rebound Attacks on AES-Like Hashing by Exploiting Related-Key Differentials. Advances in Cryptology–ASIACRYPT 2021, LNCS vol. 13090, pp. 241–271, Springer, 2021.
16. Dong, X., Zhao, B., Qin, L., Hou, Q., Zhang, S., Wang, X., Generic MitM attack frameworks on sponge constructions. Advances in Cryptology–CRYPTO 2024, LNCS vol. 14923, pp. 3–37, Springer, 2024.
17. Grover, L. K., A fast quantum mechanical algorithm for database search. Proceedings of the 28th Annual ACM Symposium on Theory of Computing, pp. 212–219, ACM, 1996.
18. Kelsey, J., Kohno, T., Herding Hash functions and the Nostradamus Attack Advances in Cryptology–Eurocrypt’2006 LNCS vol. 4004, pp.183–200, Springer-Verlag, 2006.
19. Khovratovich, D., Nikolić, I., Rotational cryptanalysis of ARX. Fast Software Encryption–FSE 2010, LNCS vol. 6147, pp. 333–346, Springer, 2010.
20. Kannwischer. M.J., Rijneveld. J., Schwabe. P., Stoffelen. K., pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4 <https://eprint.iacr.org/2019/844.pdf>
21. Knudsen, L., Truncated and Higher order differentials Fast Software Encryption–FSE 1994 LNCS vol. 1008, pp.196–211, Springer-Verlag, 1995.
22. Jakobsen, T., Knudsen, L.R., The interpolation attack on block ciphers. Fast Software Encryption–FSE 1997, LNCS vol. 1267, pp. 28–40, Springer, 1997.
23. Joux, A., Peyrin, T., Hash functions and the (amplified) Boomerang Attack. Advances in Cryptology–Crypto’2007 LNCS vol. 4622, pp.244–263, Springer-Verlag, 2007.
24. Leander, G., Abdelraheem, M.A., AlKhazaimi, H., Zenner, E., A cryptanalysis of PRINTcipher: the invariant subspace attack. Advances in Cryptology–CRYPTO 2011, LNCS vol. 6841, pp. 206–221, Springer, 2011.
25. Matsui, M., Linear cryptanalysis method for DES cipher. Advances in Cryptology–EUROCRYPT 1993, LNCS vol. 765, pp. 386–397, Springer, 1994.
26. Mendel, F., Rechberger, C., Schlaffer M., Thomsen S.S., The Rebound Attack: Cryptanalysis of Reduced Whirlpool and Grostl. Fast Software Encryption–FSE 2009 LNCS vol. 5665, pp.260–276, Springer-Verlag, 2010.
27. Mouha, N., Automatic Techniques for Hash Function and Block Cipher Cryptanalysis. PhD Thesis, Katholieke Universiteit Leuven, Belgium, 2012
28. Moradi, A., Poschman, A., Ling S., Paar C., Wang, H., Pushing the Limits: A Very Compact and a Threshold Implementation of AES. Advances in Cryptology–EUROCRYPT’2011 LNCS vol. 6632, pp.69–88, Springer-Verlag, 2011.
29. NICCS: Announcement on launching the next-generation commercial Cryptographic algorithms program (2025), <http://www.niccs.org.cn/tzgg/>
30. NIST: A statistical test suite for random and pseudorandom number generators for cryptographic applications, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-22r1a.pdf>
31. Qin, L., Hua, J., Dong, X., Yan, H., Wang, X., Meet-in-the-middle preimage attacks on sponge-based hashing. Advances in Cryptology–EUROCRYPT 2023, LNCS vol. 14007, pp. 158–188, Springer, 2023.

32. Schrottenloher, A., Stevens, M., Simplified Modeling of MITM Attacks for Block Ciphers: New (Quantum) Attacks. Advances in Cryptology–CRYPTO 2022, LNCS vol. 13509, pp. 717–747, Springer, 2022. (Permutation/sponge modeling: IACR ePrint 2023/816.)
33. Sun. S., Li. S., Zhang Z., Charlotte L., Mennink. B., Qin. Z. and Feng D., Permutation-based Hashing with Stronger (Second) Preimage Resistance: Application to Hash-Based Signature Schemes. <https://eprint.iacr.org/2025/963.pdf>
34. Wagner, D., The boomerang attack. Fast Software Encryption–FSE 1999, LNCS vol. 1636, pp. 156–170, Springer, 1999.
35. Wang. X., Lai. X., Feng D., Chen H., Yu. X., Cryptanalysis of the hash function MD4 and RIPEMD. Advances in Cryptology–Eurocrypt’2005 LNCS vol. 3494, pp.1–18, Springer-Verlag, 2005.
36. Wang. X., Yu. H., How to break MD5 and other hash functions. Advances in Cryptology–Eurocrypt’2005 LNCS vol. 3494, pp.19–35, Springer-Verlag, 2005.
37. Wang. X., Yu. H., Yin L., Efficient collision search attacks on SHA-0. Advances in Cryptology–Crypto’2005 LNCS vol. 3621, pp.1–16, Springer-Verlag, 2005.
38. Wang. X., Yin L., Yu. H., Finding collisions in the full SHA-1. Advances in Cryptology–Crypto’2005 LNCS vol. 3621, pp.17–36, Springer-Verlag, 2005.

A The Pre-computed Initial State for Each Garnet Instance

The starting state of the absorbing phase for Garnet-512 with *counter*₁ is shown below.

$$\begin{aligned}
S_0^a &= 0\text{xffdfea}80\text{e8157eb4311cceecf76770f1} \\
S_1^a &= 0\text{x486dae}24482\text{aa00aa9af0b6d22d121bd} \\
S_2^a &= 0\text{x306eda}4994\text{baba9c9f2fe6c33ab00be5} \\
S_3^a &= 0\text{x5c2173082db5ddea190a811b2f39580e} \\
S_4^a &= 0\text{xc4f59e2ab098316cf6818434230a38a3} \\
S_5^a &= 0\text{x8452307e9b13b8aeb2f34e3108b93899} \\
S_6^a &= 0\text{xd6559e703c0ffad48cd00f5e6f965b39} \\
S \leftarrow S_7^a &= 0\text{xf4139d666be07b74b9bc125c20abb2a6} \\
S_8^a &= 0\text{xf1c65f707fb1d7a7b9e97c07cb92a584} \\
S_9^a &= 0\text{xa217af9dc3b9c1690c8649adbb445f7b} \\
S_{10}^a &= 0\text{x0dc0cdb2822afe55e33cd10720739e0a} \\
S_{11}^a &= 0\text{xe060d07351d77ec8728e9fc267980396} \\
S_{12}^a &= 0\text{x8cc7e03f0c09af90e900478b90718d8f} \\
S_{13}^a &= 0\text{x80dd4e1f759fa6b9533a33dac888ed08} \\
S_{14}^a &= 0\text{x454bf20854edf819d626dbde1d3bbe21} \\
S_{15}^a &= 0\text{xa8dd28acb21d71971a9fe56ae1482d24} .
\end{aligned}$$

The starting state of the absorbing phase for Garnet-512 with $counter_2$ is shown below.

$$\begin{aligned}
S_0^a &= 0xbaa89618b47c4eee2ebf3abde3fb6116 \\
S_1^a &= 0xbe0b0c92abdb3cda077ad97232ad617b \\
S_2^a &= 0x7e9b096bd65a52495950cdfba838e796 \\
S_3^a &= 0x83b61edc223526eeaf4541046e765fac \\
S_4^a &= 0xbca55cfac429c8605c62cda23fe20d1e \\
S_5^a &= 0x05881a1f883f16ff7f579965691de336 \\
S_6^a &= 0xf38538686ae0eb4e9c15f5b5b93ff854 \\
S \leftarrow S_7^a &= 0xd9c35429b75fe28afd0a7691efac606a \\
S_8^a &= 0xcab740a879db30cd292b1d791af6aec8 \\
S_9^a &= 0x417579fcb695694ed3b38b44b3c9d1eb \\
S_{10}^a &= 0xf727a8e7871d5fb946538867280ecd92 \\
S_{11}^a &= 0x36b161e3e03a98e7f673c20f669d63ac \\
S_{12}^a &= 0x6eefd4f36d1bd0a0566cc32dd61c5169 \\
S_{13}^a &= 0x0e3adafb2f9ad32eb095703d0d4e2db5 \\
S_{14}^a &= 0x25ec1ece9c94d6b58a5c72a07dd7088e \\
S_{15}^a &= 0x616156ebe4b971e61e79e0199d994a74 .
\end{aligned}$$

The starting state of the absorbing phase for Garnet-512 with $counter_3$ is shown below.

$$\begin{aligned}
S_0^a &= 0x1b71dce00e40b424d1d22a3f4aa75a20 \\
S_1^a &= 0x314be1c5b9e8a5635581ac2728f164b8 \\
S_2^a &= 0xbb81c51d35e9987764fc5ecb7b6692e3 \\
S_3^a &= 0x58858dc92c74f6fc5f698d2be061907c \\
S_4^a &= 0x998e7bc1d9162e8db9d3cb972aff6aeb \\
S_5^a &= 0x89c421090dbb2912f26c4bf35ad8fdd2 \\
S_6^a &= 0x28c09b38e704db42aaaa25b5b302ff3a \\
S \leftarrow S_7^a &= 0x1d66f67cad5dd3350ea9d84402f9ac85 \\
S_8^a &= 0x2021b612913c9fd99d3b80acfdced1d2 \\
S_9^a &= 0xd64db88cd67650bde44220a3cce4bca \\
S_{10}^a &= 0x56ee5b87799345f20862a4e1ee82c2d0 \\
S_{11}^a &= 0xe438c2932439ce59181c6b55876bed3a \\
S_{12}^a &= 0x92f2db933348b7b1e1bdfaa580c622d7 \\
S_{13}^a &= 0xcf876eb66883cf9b93ac5cb01862ed75 \\
S_{14}^a &= 0x9ac1852305b140af5b80980b7654ae96 \\
S_{15}^a &= 0x246f7e73910804929b89523fa05230c5 .
\end{aligned}$$

The starting state of the absorbing phase for Garnet-512 with $counter_4$ is shown below.

$$\begin{aligned}
S_0^a &= 0xa4b8433d99d491edce4f49d38fe0830a \\
S_1^a &= 0x73317b70d2b67ab34ca1daedfbff54fa \\
S_2^a &= 0xb9dbff2ff29f4466a70e9223fbb3de14 \\
S_3^a &= 0x01df8af21eceedf543cd103aecc958c5 \\
S_4^a &= 0x6a32489ae800f8344d165d1cd86cc8d4 \\
S_5^a &= 0x2224c4159c4951cc05f9522bf88d6a96 \\
S_6^a &= 0x6db6b4e6b6337716a1d714b1ac3c0e6c \\
S \leftarrow S_7^a &= 0x681866d4c5ca710d661d5963f16a276a \\
S_8^a &= 0xf68990ae9cf6618e57133677381d8b01 \\
S_9^a &= 0xf6e8f188dc038a494ce6b5a186f5b5ce \\
S_{10}^a &= 0x577a951517e7a95a5e9cf253dcd91cfd \\
S_{11}^a &= 0x40dfc12faa2d68d592de402ccb02f387 \\
S_{12}^a &= 0xd82968bd47091988ab13fd4cbb26505d \\
S_{13}^a &= 0x5d459579a983f285f72e1a744a39c552 \\
S_{14}^a &= 0xcae4079778f0af793a8a941b569abc22 \\
S_{15}^a &= 0xe45f3a0f93bdfaa66fe36fc7a21df058 .
\end{aligned}$$

The starting state of the absorbing phase for Garnet-768 with $counter_5$ is shown below.

$$\begin{aligned}
S_0^a &= 0x900e7220fbc0a2a9a81adffbd7e4ecc5 \\
S_1^a &= 0xb2c6cf5cb985a343452112cbd38f687b \\
S_2^a &= 0x02532ef420b221ba6f32196521006231 \\
S_3^a &= 0x8c0aa10a77562b54554af4c708f01b5b \\
S_4^a &= 0x3d9b8469af4e2f62253b2aa5752073ee \\
S_5^a &= 0x9cc56b68c0bf330dfb759b1e991915e9 \\
S_6^a &= 0x87d6ad22e0991acf1944e4f280f0f9d6 \\
S \leftarrow S_7^a &= 0x915e00c3ed5649034d6c7fdeb90a1d16 \\
S_8^a &= 0xba2bc2c6d97c6bd87d345c648773c306 \\
S_9^a &= 0x08b97165f30fa3890fc33894dffdd793 \\
S_{10}^a &= 0x39051f12faaa087ea75d9c9cbc8de0df \\
S_{11}^a &= 0xc4eeddfb4281a22e8d7bea10122be937 \\
S_{12}^a &= 0x7c1e61fd2cb3f5699b6b038a7b8787bc \\
S_{13}^a &= 0xffc1cf7b127b4c8ddd9fc604e7f3f995 \\
S_{14}^a &= 0x1ab40f1cc68e821d2abeb5fa34dec371 \\
S_{15}^a &= 0xf69fd9975760832e3b29e17602a41b9f .
\end{aligned}$$

The starting state of the absorbing phase for Garnet-1024a with $counter_6$ is shown below.

$$\begin{aligned}
S_0^a &= 0xae3e63c412c399c76d6794b2b44ce079 \\
S_1^a &= 0x4ba73a9694578b7b3313b546b6b97c6a \\
S_2^a &= 0xf1732c187b5674481ffe7cde7b97f757 \\
S_3^a &= 0xf306671bb4c3825d3ca91ae242531b1b \\
S_4^a &= 0xf30f2348484e1307f93197cdf02b501 \\
S_5^a &= 0xfabc1c4d4fc43e64ea2ca2425a036ee4 \\
S_6^a &= 0x706de8c85e5c85ef11d429de5a6f596c \\
S \leftarrow S_7^a &= 0xf768093617b362bd0c62de986f621216 \\
S_8^a &= 0xcb8d620f0de9ab02e9bb5cabecfdc033 \\
S_9^a &= 0xda3f7bac0b8a4288f3786fd62acc7f \\
S_{10}^a &= 0x29c38914c6d955225025d51190f4aef1 \\
S_{11}^a &= 0xd35f234f7bb89d5cfa559a587be6b290 \\
S_{12}^a &= 0x9b5e142fb1b033fb0e7454120469cf3c \\
S_{13}^a &= 0xc5b11c6410d4ed09e259137329632770 \\
S_{14}^a &= 0x9cf6fef3c9cf33b33616a45fa80ee0b \\
S_{15}^a &= 0xf90e95fde810f135194ea6997ef891f9 .
\end{aligned}$$

B The Absorbing Patterns in the Other Garnet Instances

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-512/w=r=640 bit

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-512/w=r=768 bit

S_0	S_1	S_2	S_3
S_4	S_5	S_6	S_7
S_8	S_9	S_{10}	S_{11}
S_{12}	S_{13}	S_{14}	S_{15}

Garnet-512/w=r=896 bit

Table 18. NIST SP 800-22 statistical randomness screening of the Garnet digest streams (CLAASP STS; 1000×10^6 bits per stream, all 15 families = 188 sub-tests, $\alpha = 0.01$). Each cell is the number of sub-tests passed for the given instance and message-sample ensemble.

Instance	rnd (random)	ctr (counter)	ofb (feedback)	low (low weight)	high (high weight)
Garnet-512	188/188	188/188	188/188	188/188	188/188
Garnet-768	188/188	188/188	188/188	188/188	188/188
Garnet-1024a	188/188	188/188	188/188	188/188	188/188

C Empirical Statistical Randomness Tests

We performed an implementation-level statistical randomness test of the digest streams of the three Garnet algorithms covering all instances with the NIST SP 800-22 Statistical Test Suite (its 15 families expand to 188 sub-tests), as implemented in the automated CLAASP tooling [3,30]. We follow the standard randomness testing of digest streams methodology: for each instance a large number of messages is generated, each message is hashed, and the resulting digests are concatenated into a single digest stream on which the full suite is run. To exercise the hash on qualitatively different inputs, the messages are produced under *five* complementary deterministic ensembles, and every (instance, ensemble) stream is screened independently. Such tests are necessary implementation-level sanity checks; they are not a proof of pseudorandomness and do not replace the structural cryptanalysis of the permutation or the ideal-model analysis of the construction mode.

Methodology and reproducibility. The streams were produced by the submitted reference implementation. Each message block is hashed and the 512/768/1024-bit digests are concatenated under the five message-sample ensembles: **rnd** (pseudo-random `splitmix64`-derived messages, probing general randomness); **ctr** (a 64-bit counter in an otherwise zero block, probing diffusion of strongly structured inputs); **ofb** (output feedback, i.e., each digest is fed back as the next message, probing the hash as an iterated map); **low** (messages enumerated in increasing Hamming weight 0, 1, 2, ..., i.e. very sparse inputs); and **high** (the bitwise complement of **low**, i.e. very dense inputs). The NIST configuration used the high-confidence setting of 1000 sequences of 10^6 bits each (10^9 bits per stream), binary input, all 15 families, at significance level $\alpha = 0.01$; the population threshold is $\geq 981/1000$ for the ordinary tests, with an additional 10-bin p -value uniformity check ($\geq 10^{-4}$). Across the three instances and five ensembles this screens 15 digest streams totalling 1.5×10^{10} bits.

Results. Every one of the 15 digest streams passes the entire suite: all 188 sub-tests are satisfied for each of the three instances under each of the five message-sample ensembles, that is, 2820/2820 sub-tests in aggregate, with *no* sub-threshold entry of any kind, neither a population-proportion shortfall nor a p -value-uniformity flag. The per-stream results are shown in Table 18. In particular the structurally extreme ensembles, i.e., the low- and high-Hamming-weight messages and the output-feedback iteration, are statistically indistinguishable from the pseudo-random ensemble at this resolution.

Complementary avalanche check. As an independent first-order check on the same reference implementation, the strict avalanche criterion (SAC) was measured over 160,000 single-bit-flip trials per instance. The mean output Hamming distance equals 50.0019%, 49.9977% and 50.0002% of the digest length for Garnet-512, Garnet-768 and Garnet-1024a respectively (ideal 50%), and the observed spread matches the ideal $\sqrt{n}/2$ standard deviation (11.33 vs. 11.31, 13.87 vs. 13.86, and 16.04 vs. 16.00 bits).

Consequently, under five complementary message ensembles and the high-confidence 1000×10^6 -bit configuration, the tested reference implementations of Garnet-512, Garnet-768 and Garnet-1024a exhibit no detected statistical non-randomness in their digest streams. This is empirical evidence that the reference implementation has no elementary distributional defect, it is not a bound on collision, preimage, second-preimage, quantum, differential, rebound, or length-extension security.